

Prim vs. Kruskal: Uma Avaliação Empírica de Vantagens e Desvantagens no Problema da Árvore Geradora Mínima

Guilherme da Silva Ferraz¹

¹ Graduando em Ciências da Computação/ICET/CUA

Universidade Federal do Mato Grosso (UFMT) - Barra do Garças - MT – Brasil

guilherme.ferraz1@sou.ufmt.br

Resumo: Este trabalho acadêmico apresenta a aplicação de dois dos principais métodos da teoria dos grafos para lidar com árvores geradoras mínimas: os algoritmos de Prim e Kruskal. A análise utiliza dados reais a fim de demonstrar, na prática, as diferenças de eficiência e o comportamento de ambas as abordagens sob diferentes cenários e densidades de grafos. A implementação de ambos os algoritmos foi desenvolvida na linguagem de programação C. Os resultados experimentais revelaram que, embora ambos garantam o mesmo custo total, o tempo de CPU variou drasticamente dependendo de como lidam com grafos esparsos e densos. O algoritmo de Prim foi mais rápido no grafo menor (Nova York), enquanto o algoritmo de Kruskal mostrou-se mais escalável na instância massiva (Califórnia).

1. Introdução

Um grafo G é conceitualmente uma estrutura matemática abstrata definida por $G = (V, E)$, onde V é conjunto de todos os vértices e E é conjunto de todas as arestas que compõem esse grafo. Na ciência da computação, eles são amplamente utilizados para modelar relacionamento entre entidades com naturezas distintas. Redes de computadores, redes sociais, rotas aéreas, sistemas de transporte, por exemplo, podem ser representados como grafos.

Um exemplo prático ocorre no planejamento logístico de uma transportadora que esteja buscando otimizar suas rotas de distribuição a fim de diminuir os custos operacionais. Dessa maneira, as cidades podem ser representadas por vértices e as estradas por arestas ponderadas, cujos pesos correspondem às distâncias entre as cidades. Uma possível aplicação consiste na análise da ampliação do número de filiais em cidades do interior, considerando que as demais sedes sejam em capitais estaduais e que a depender do estado, muitas cidades se encontram a grandes distâncias das filiais. O problema de interligar todos os pontos com o menor custo possível e sem formar ciclos - um ciclo em um grafo é um caminho fechado, onde o percurso começa e termina no mesmo vértice - se traduz diretamente na busca por uma Árvore Geradora Mínima.

Este trabalho propõe um estudo experimental entre dois dos principais métodos da literatura: os algoritmos de Prim e Kruskal. A análise utiliza dados reais a fim de demonstrar na prática, as diferenças de eficiência e o comportamento de ambas as abordagens sob diferentes cenários e densidades de grafos.

2. Problema da Árvore Geradora Mínima (AGM)

Em projeto de circuitos eletrônicos, muitas vezes, é necessário que os pinos de vários componentes se tornem eletricamente equivalentes, o que é conseguido ligando-os uns aos outros. Para interconectar um conjunto de n pinos, podemos usar um arranjo de $n - 1$ fios, cada qual conectando dois pinos. De todos os arranjos possíveis, aquele que utiliza a mínima quantidade de fio é normalmente o mais desejável [Cormen 2012].

Conforme explicitado por Cormen, podemos modelar esse problema de fiação como um grafo conexo não-orientado $G = (V, E)$, onde V é o conjunto de pinos, E é o conjunto de interconexões possíveis entre pares de pinos e, para cada aresta $(u, v) \in E$, temos um peso $w(u, v)$ que especifica o custo (a quantidade necessária de fio) para conectar u e v . Então a partir desse grafo, desejamos encontrar um subconjunto $T \subseteq E$ que minimize o peso total e que conecte todos os vértices de um grafo sem formar um ciclo.

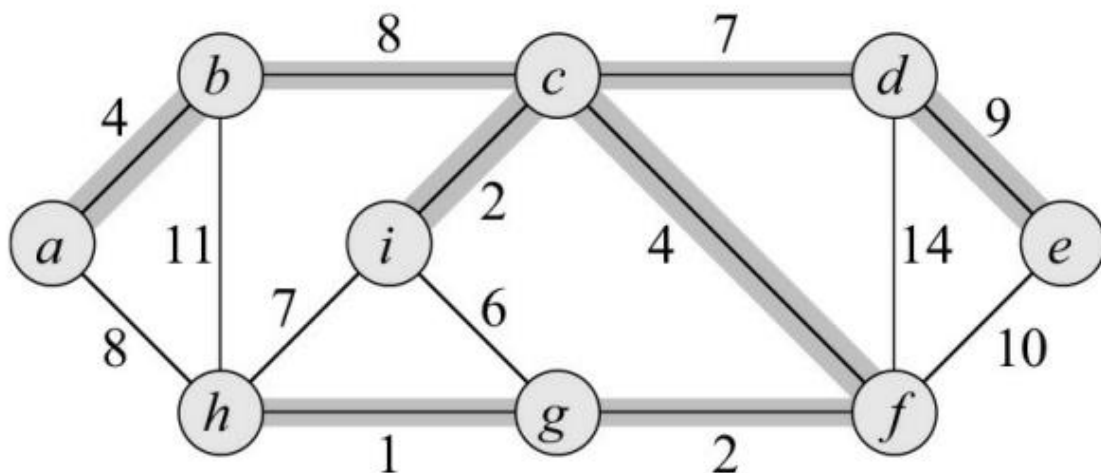


Figura 1. Uma árvore geradora mínima para um grafo conexo. Os pesos nas arestas são mostrados, e as arestas em uma árvore geradora mínima estão sombreadas. O peso total da árvore mostrada é 37.

Na figura 1, é ilustrado uma das possíveis AGM para esse grafo. Se removermos a aresta (b, c) e a substituirmos pela aresta (a, h) produziremos outra árvore geradora com peso 37.ca

3. Algoritmos Gulosos

O problema das AGM pode ser resolvido por meio de algoritmos gulosos, isto é, algoritmos que sempre fazem a escolha aparentemente mais vantajosa no momento, na esperança de alcançar uma escolha globalmente ótima.

No contexto de Árvore Geradoras Mínimas, a estratégia gulosa baseia-se na regra de seleção de arestas seguras. Durante esse processo, mantém-se um subconjunto S de arestas tal que S é um subconjunto de uma árvore geradora mínima, onde uv é uma aresta segura para S se pode ser adicionada sem violar a propriedade de S . Assim, o algoritmo avança adicionando uma aresta segura por vez. Então, para determinar quais arestas são seguras, utiliza-se o conceito de corte em um grafo, isto é, um corte de um grafo é uma partição de seus vértices em dois conjuntos disjuntos. Segundo Wayne & Sedgewick (2011), dado qualquer corte em um grafo ponderado por arestas (com todos os pesos das arestas distintos), a aresta de cruzamento de peso mínimo está na árvore geradora mínima (AGM) do grafo.

3.1. Prim

O algoritmo de Prim foi originalmente desenvolvido pelo matemático tcheco Vojtěch Jarník(1930). Anos mais tarde, ele foi redescoberto independentemente e publicado por Robert C. Prim(1957), por esse motivo, o algoritmo é frequentemente denominado Algoritmo de Prim-Jarník na literatura científica. Trata-se de um algoritmo guloso e sua ideia básica é iniciar a formação da árvore geradora de custo mínimo a partir de um vértice raiz arbitrário. Em cada iteração do algoritmo, a árvore cresce por meio da adição de um novo vértice [Gomes 2022]. Dessa maneira, o algoritmo de Prim sempre irá buscar pela aresta que adicione o menor peso possível para a árvore, repetindo esse processo até que todos os nós estejam interconectados.

Como apontado por Chen (2023), o Prim pode ser ineficiente ao lidar com grafos esparsos — aqueles para os quais $|E|$ é muito menor que $|V|^2$ —, o que requer o uso de otimização de heap. Consequentemente, isso aumenta a complexidade estrutural da implementação, embora otimize o tempo de execução para $O(E \log V)$. Diante dessa necessidade, foi implementado o heap mínimo (min-heap) para gerenciar a fila de prioridade do algoritmo.

```

1 Prim(G,r):
2   para cada  $v \in V(G)$  faça
3      $p[v] = \infty$ 
4      $pai[v] = -1$ 
5    $p[r] = 0$ 
6   Constrói heap mínimo  $A$  com  $V(G)$  (com base em  $p$ )
7    $S = \emptyset$ 
8   enquanto  $|A| > 1$  faça
9      $u = RetiraMin(A)$ ; Refaz heap
10     $S = S \cup \{u\}$ 
11    para  $v \in adj(u)$  faça
12      se  $(v \in A)$  e  $(p[v] > p(u, v))$  então
13         $p[v] = p(u, v)$ 
14         $pai[v] = u$ ; Refaz heap

```

Figura 1. Pseudocódigo do algoritmo de prim.

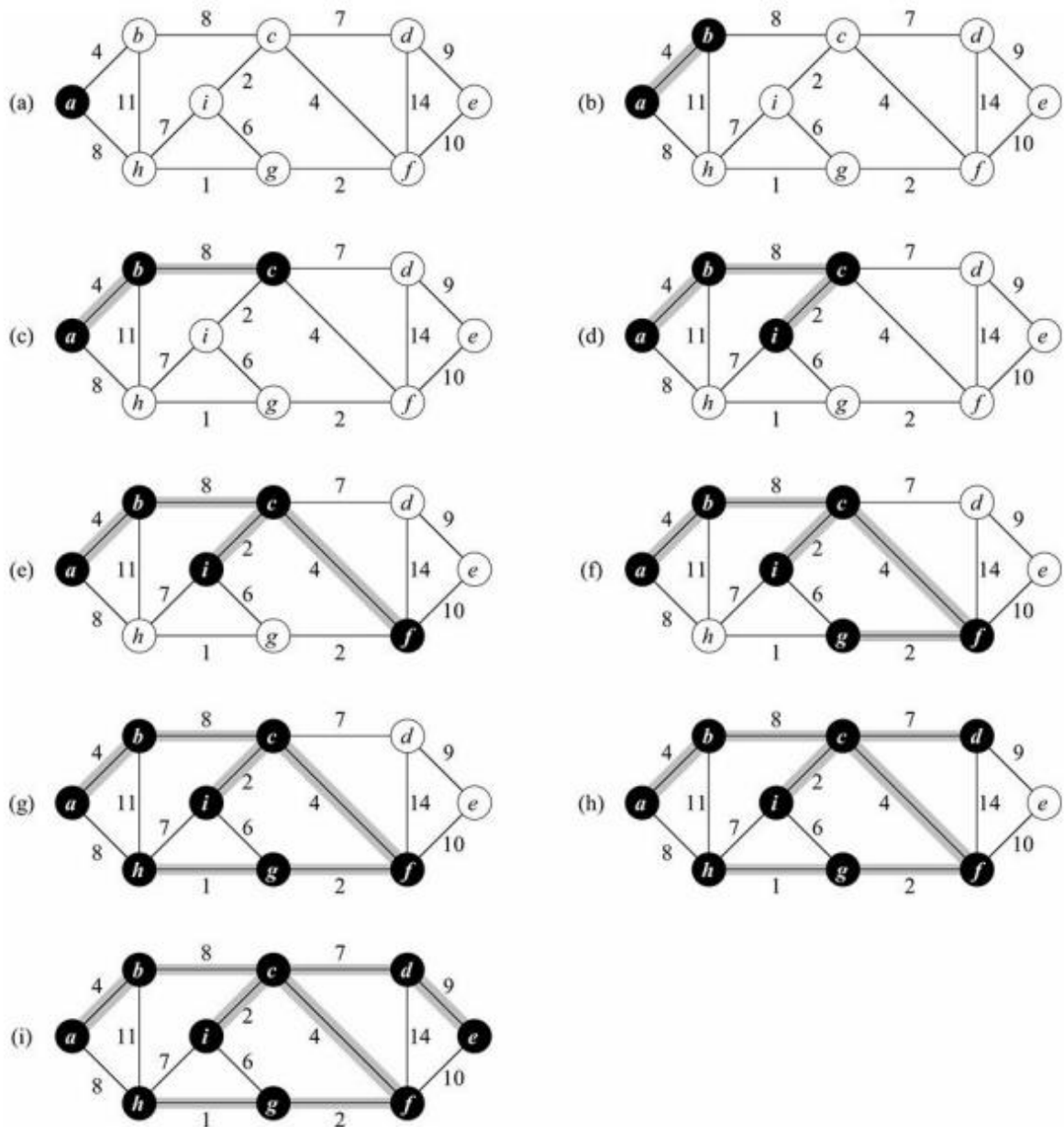


Figura 2. Evolução da árvore através do método de Prim. CORMEN et al., 2012, Capítulo 23 - Árvores Geradoras Mínimas).

3.2. Kruskal

O algoritmo de Kruskal foi desenvolvido pelo matemático Joseph Kruskal em 1956. Este método encontra uma floresta geradora de custo mínimo em grafos valorados. Se o grafo for conexo, então o algoritmo encontra uma árvore geradora de custo mínimo [Gomes 2022]. Kruskal adota a premissa de que eventualmente todos os grafos se tornarão uma AGM se todas as arestas forem selecionadas gradualmente — priorizando sempre as de menor peso —, e desde que não formem um ciclo. Toda vez que um ciclo é detectado, as arestas em questão são descartadas e não poderão ser inseridas novamente na solução. Kruskal (1956) definiu os seguintes passos para a implementação do algoritmo:

1. Ordena-se todas as arestas do grafo de forma crescente com base em seus respectivos pesos.
2. Percorre-se a lista ordenada, adicionando a aresta de menor peso disponível à árvore em construção, sob a condição estrita de que ela não crie um circuito fechado (ciclo).
3. Repete-se esse processo de varredura e inserção até que todos os vértices do grafo estejam conectados por uma única componente conexa.

```
1 Kruskal(G):  
2   para  $v \in V(G)$  faça  
3     MAKE-SET( $v$ )  
4   Ordena arestas de  $E(G)$  por  $p$  não decrescente  
5    $S = \emptyset$   
6   para cada  $(u, v) \in E(G)$  em ordem não-decrescente faça  
7     se FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ ) então  
8        $S = S \cup \{(u, v)\}$   
9       UNION( $u, v$ )  
10  retorne  $S$ 
```

Figura 3. Pseudocódigo do algoritmo de kruskal.

Como pode ser observado no pseudocódigo anteriormente, o algoritmo de kruskal necessita ordenar inicialmente as arestas de acordo com seu peso. Em grafos muito densos, essa etapa pode aumentar significativamente o tempo de execução, tornando o algoritmo potencialmente mais lento do que o Prim. A fim de minimizar esse problema, foi utilizado o algoritmo QuickSort, considerando que sua complexidade no melhor caso é de $O(n \log n)$ e no pior caso, $O(n^2)$.

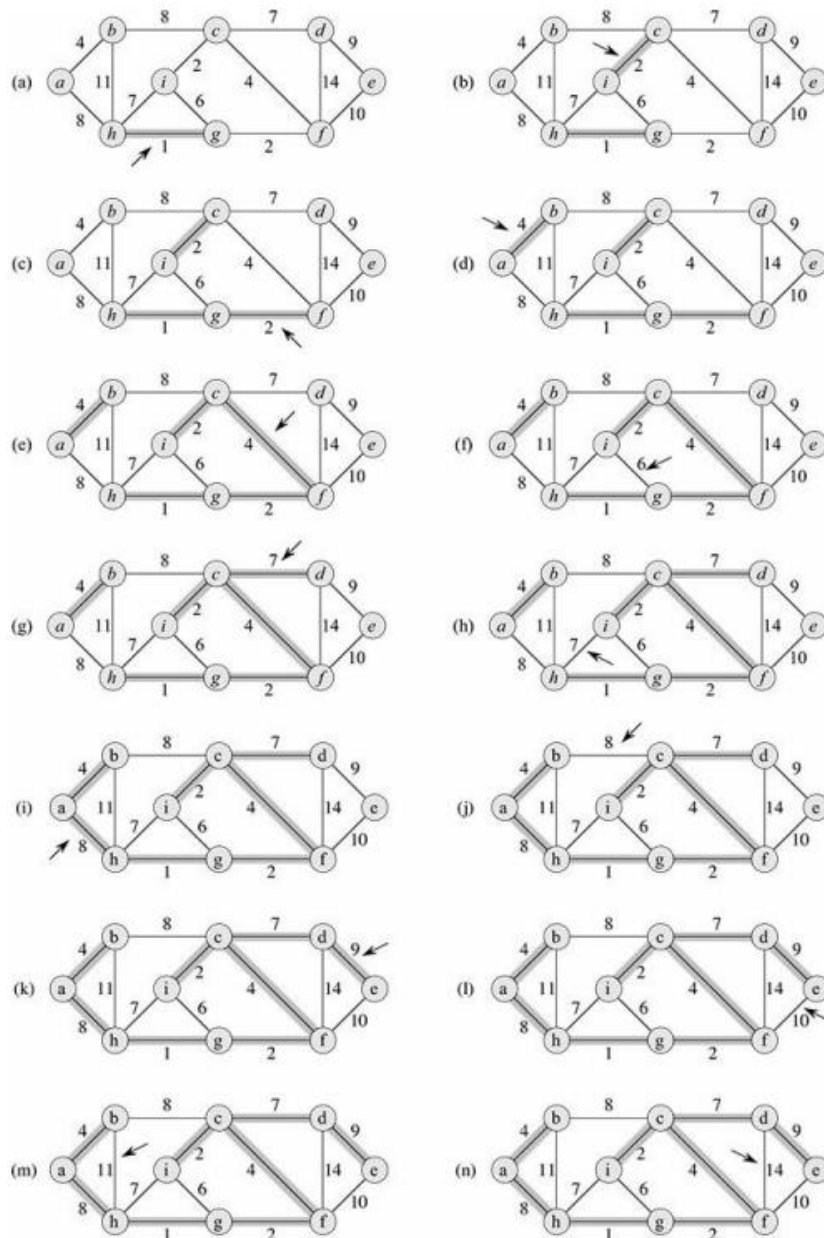


Figura 4. Evolução da árvore através do método de Kruskal. Fonte: CORMEN et al., 2012, Capítulo 23 - Árvores Geradoras Mínimas).

4. Metodologia

A análise experimental comparativa entre os algoritmos de Árvore Geradora Mínima (AGM) de Kruskal e Prim foi estruturada em três etapas principais:

1. Implementação das estruturas

Os algoritmos foram desenvolvidos em arquivos separados (kruskal.c e prim.c), adotando boas práticas de programação modular na linguagem C. O grafo foi representado computacionalmente por meio de estruturas de **Listas de Adjacência** para

otimizar o espaço em memória. Cada programa foi construído com as seguintes particularidades estruturais:

- Algoritmo de Prim: Utilizou uma fila de prioridades gerenciada por uma estrutura de Heap Mínimo (min-heap) desenvolvida do zero, garantindo a extração eficiente do vértice de menor chave a cada iteração.
- Algoritmo de Kruskal: Utilizou o método de ordenação QuickSort com pivô aleatório para ordenar o vetor global de arestas de forma crescente. O controle de ciclos e conectividade dos componentes florestais foi gerenciado eficientemente através da estrutura de conjuntos disjuntos Union-Find com compressão de caminhos.

2. Preparação de dados

Para esta etapa, foram utilizados dois conjuntos de dados massivos e reais provenientes do desafio de desafios de rotas do DIMACS:

- USA-road-d.NY.gr: Malha rodoviária da cidade de Nova York.
- USA-road-d.CAL.gr: Malha rodoviária do estado da Califórnia

3. Execução dos experimentos:

Para cada instância de grafo rodoviário, foram realizados os seguintes procedimentos automatizados:

- Leitura dos arquivos .gr e carregamento do grafo em memória.
- Execução isolada do algoritmo de AGM (Kruskal ou Prim) a partir de um vértice de origem predefinido.
- Contabilização do Custo Total da árvore geradora gerada. No Kruskal, o custo foi acumulado dinamicamente a cada união aceita; no Prim, foi computado na pós-execução, varrendo o vetor de pais gerado.
- Exportação automática dos relatórios de benchmark contendo o nome do arquivo, o custo final e o tempo de execução para os arquivos de log `saidaKruskal.txt` e `saidaPrim.txt`.

4. Medição de tempo dos experimentos:

O tempo de execução de cada algoritmo foi medido utilizando a função `times()` da biblioteca `<sys/times.h>` através do mapeamento da estrutura `struct tms`. Essa abordagem permitiu monitorar exclusivamente o tempo de CPU consumido pelo processo (desconsiderando atrasos de I/O do sistema operacional), dividindo a diferença de clock ticks pela constante de configuração do sistema (`_SC_CLK_TCK`), assegurando rigor científico e reprodutibilidade nos resultados de performance.

Para a execução dos testes, foi utilizada um notebook Dell Inspiron |15-I1300-U60P com as seguintes especificações: processador Intel Core i5 de 13a geração, 16 GB de memória RAM, SSD de 1 TB e rodando no sistema operacional Linux. Para executar o código deve-se utilizar o comando *make* e depois “./experiment_prim” e “./experiment_kruskal”.

5. Resultados e Discussões

Nesta seção serão apresentados os resultados dos testes a fim de analisar a performance dos algoritmos de Prim e Kruskal aplicados aos grafos da rede rodoviária de NY (New York) e CAL (Califórnia e Nevada) do 9º Desafio de Implementação do DIMACS. Para essa análise, foram considerados a precisão no cálculo do custo total da Árvore Geradora Mínima (AGM) e o tempo de processamento (CPU) requerido por cada algoritmo para a execução da tarefa.

Como pode ser observado na Tabela 2, Kruskal e Prim produziram AGMs com custos totais idênticos em ambos os grafos analisados. Para o grafo USA-road-d.CAL.gr, o custo total obtido foi de 4.270.584.202, enquanto para o grafo USA-road-d.NY.gr o valor encontrado foi de 261.159.288. Esses resultados demonstram que ambos os algoritmos gulosos foram capazes de encontrar a mesma solução ótima, independentemente do modo como realizam a varredura do grafo.

Embora tenham alcançado os mesmos resultados em relação ao custo total em ambos os grafos, o tempo de CPU apresentou variações significativas entre as implementações. Como evidenciado pelo gráfico da figura 6, o algoritmo de kruskal obteve um melhor resultado ao processar o grafo de maior instância (CAL), enquanto Prim obteve um melhor tempo de execução ao processar o grafo de menor instância (NY). O fato de Prim ter obtido um resultado inferior ao lidar com uma instância tão grande e de natureza esparsa como o Cal corrobora com o estudo de Chen (2023), ao afirmar que esse algoritmo perde eficiência ao lidar com grafos esparsos. Ademais, embora kruskal necessite de uma ordenação inicial – o que pesou contra seu desempenho na instância NY devido à densidade relativa ligeiramente superior deste grafo — com aproximadamente 2,77 arestas por vértice, em comparação com 2,46 arestas por vértice no grafo CAL —, esse fator isoladamente deve ser analisado cuidadosamente levando em consideração a eficiência do Min-Heap no algoritmo de Prim em cenários controlados. Como citado por Cormen (2012), o tempo de execução do algoritmo de Prim dependerá de como implementaremos a fila de prioridades Q. Ao implementarmos a fila de prioridade Q como um heap mínimo binário, o autor demonstra que obteremos uma complexidade de tempo total de $O(lg V)$. Na prática, isso poderia explicar a vantagem de Prim para o grafo NY: como essa

instância é menor, o volume de vértices reduzido não sobrecarrega o heap, permitindo que as operações rodem direto na memória cache do processador.

Tabela 1. Dados de cada instância

Instância	Vértices (V)	Arestas (E)	$\frac{E}{V}$
CAL	1.890.815	4.657.742	2,77
NY	264.346	733.846	2,46

Tabela 2. Resultados dos testes.

Arquivo	USA-road-d.CAL.gr		USA-road-d.NY.gr	
AGM	Custo total	Tempo Total (segundos)	Custo total	Tempo Total (segundos)
Kruskal	4270584202	1.1700	261159288	0.2100
Prim	4270584202	1.2800	261159288	0.1300

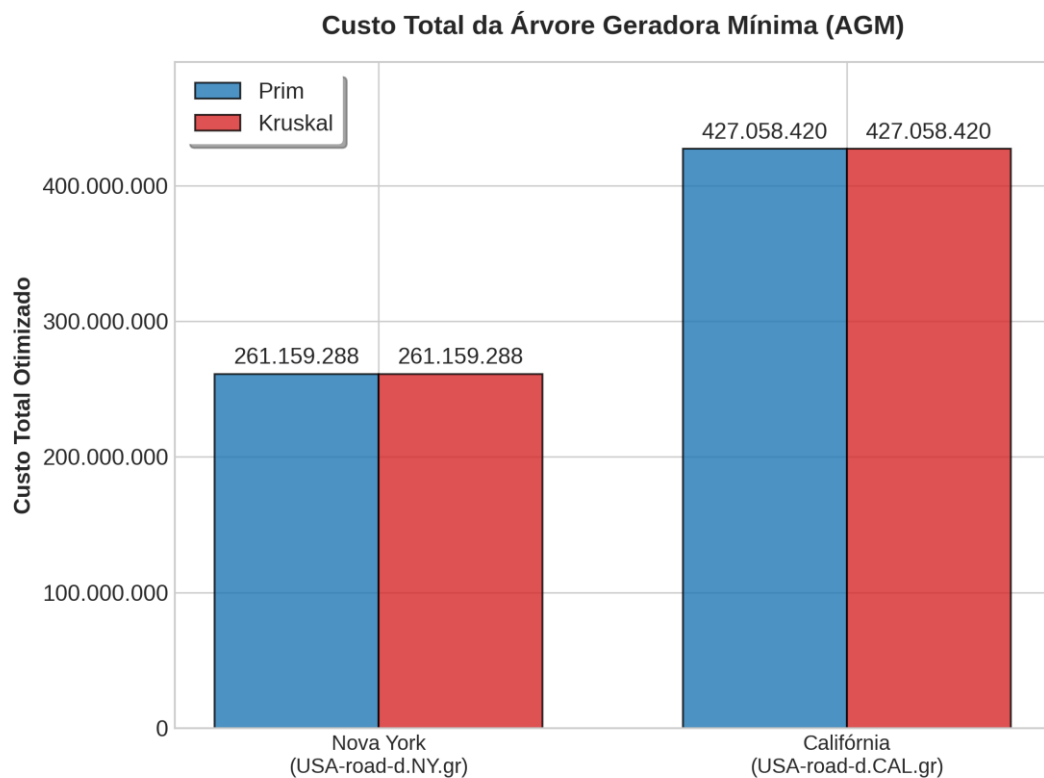


Figura 5. Gráfico comparativo do custo total dos algoritmos de Kruskal e Prim.

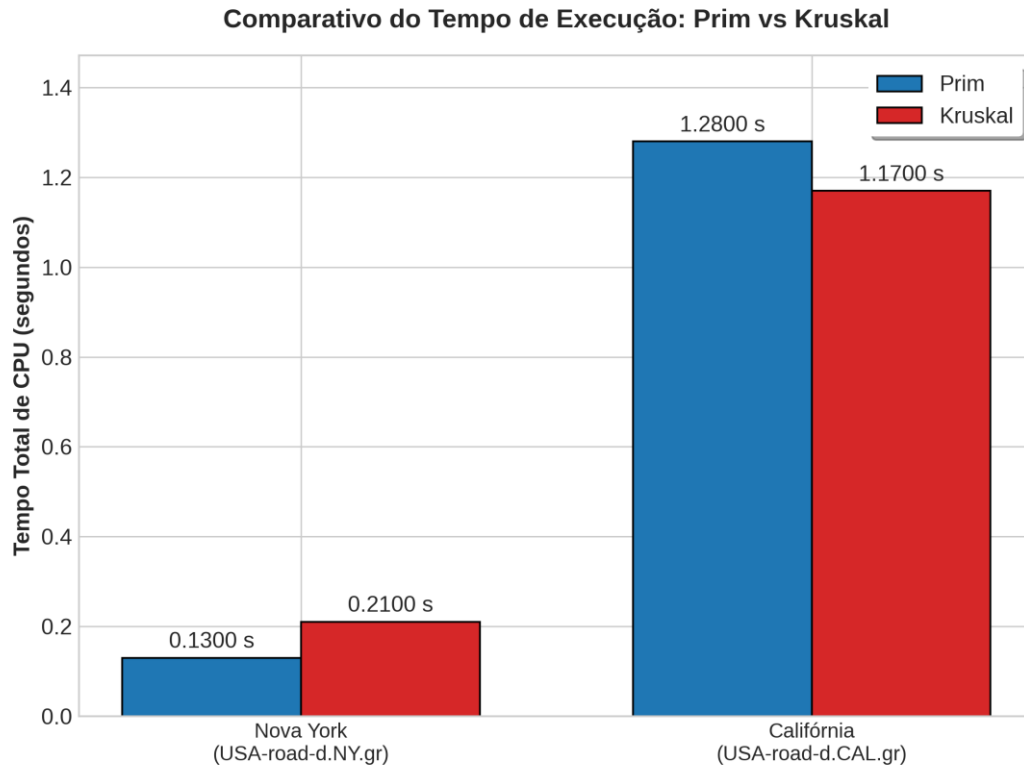


Figura 6. Gráfico comparativo do tempo de execução dos algoritmos de Prim e Kruskal.

6. Considerações Finais

Este estudo cumpriu com êxito o objetivo de analisar e comparar experimentalmente o desempenho dos algoritmos de Prim e Kruskal na determinação da Árvore Geradora Mínima (AGM). A utilização de instâncias reais da malha rodoviária americana forneceu uma base de testes sólida para avaliar o comportamento de ambas as estruturas em cenários com grafos esparsos e grandes volumes de dados. No contexto acadêmico, esse estudo contribui para a compreensão dos estudantes sobre a eficiência dos algoritmos analisados.

O algoritmo de Prim mostrou-se mais eficiente no grafo de Nova York. Apesar de ser uma diferença de apenas 0,08 segundos em termos absolutos, esse resultado evidencia como o Prim consegue responder mais rápido em estruturas menores. Em contrapartida, Kruskal obteve melhores resultados ao lidar um grafo bastante esparsos e massivo. Os resultados deste estudo são consistentes com os apresentados por Corrêa e Rangel (2023). Os autores identificaram que o algoritmo de Kruskal foi superior em cenários esparsos. Como as malhas rodoviárias testadas são tipicamente esparsas, o Kruskal tirou proveito dessa característica estrutural ao

processar o grafo da Califórnia (CAL) de forma mais eficiente que o Prim, cujo desempenho acabou por ser limitado pelo tamanho da fila de prioridades.

Por fim, o estudo demonstrou que a escolha do melhor algoritmo dependerá diretamente da natureza do grafo.

7. Referências Bibliográficas

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Algoritmos: Teoria e Prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.

DASGUPTA, S.; PAPADIMITRIOU, C.; VAZIRANI, U. (2006). **Algorithms**. McGraw-Hill Higher Education.

SEdgeWICK, R.; WAYNE, K. (2011). *Algorithms*. 4th ed. Addison-Wesley Professional.

GOMES, Paulo César Rodacki. Grafos: conceitos fundamentais, algoritmos e aplicações [livro eletrônico]. Blumenau: Editora do Instituto Federal Catarinense, 2022. PDF. ISBN 978-65-88089-12-5. Disponível em:

<https://editora.ifc.edu.br/wp-content/uploads/sites/39/2022/11/Grafos-versao-final.pdf>.

Acesso em: 12 jun. 2026.

CHEN, Jiayu. The analysis and application of Prim algorithm, Kruskal algorithm, Boruvka algorithm. *Applied and Computational Engineering*, v. 19, n. 1, p. 84-89, out. 2023. Disponível em:

https://www.researchgate.net/publication/381997773_The_analysis_and_application_of_Prim_algorithm_Kruskal_algorithm_Boruvka_algorithm. Acesso em: 12 jun. 2026.

UNIVERSITÀ DI ROMA LA SAPIENZA. Dipartimento di Ingegneria Informatica, Automatica e Gestionale (DIAG). 9th DIMACS Implementation Challenge - Shortest Paths. Roma, [2006]. Disponível em:

<https://www.diag.uniroma1.it/challenge9/download.shtml>. Acesso em: 4 jun. 2026.

RANGEL, Diego Bourguignon & CORRÊA, Arthur de Moraes Andrade. (2023, July 7). Implementação do Algoritmo de Kruskal e PRIM. Even3 Publicações.